Part B-Comparative Analysis of Recurrent Neural Architectures for Sentiment Classification

Manjusha Deore

1 Introduction

The objective of this assignment is to study and compare the effectiveness of different recurrent neural network architectures, namely vanilla recurrent neural networks (RNNs), long short-term memory networks (LSTMs), and attention-augmented variants, for binary sentiment classification. This report details the implementation, challenges, and performance metrics of these models using various word embedding strategies.

2 Dataset Description

Large Movie Review Dataset: This is a dataset for binary sentiment classification containing substantially more data than previous benchmark datasets. We provide a set of 25,000 highly polar movie reviews for training, and 25,000 for testing. There is additional unlabeled data for use as well. Raw text and already processed bag of words formats are provided. For the purpose of these experiments, we utilize the version 1.0 of this dataset, specifically targeting the labeled polarized reviews to train models to output a single sentiment label.

3 Question B1: Preprocessing

The preprocessing phase is critical to reduce vocabulary redundancy and prepare sequences for recurrent processing.

- **Normalization**: Standard techniques including lowercasing and punctuation stripping were applied.
- **Tokenization**: Word-to-integer mapping was performed with a vocabulary limit.
- **Sequence Handling**: Optimized for RNNs using **Pre-Padding** and **Pre-Truncating**.

Execution Log Details:

- Loading data from *imdb.npz*.
- Training/Testing samples: 25,000 each.
- Vocabulary Size: 10,000.
- Fixed Sequence Length: 150.
- Sample Tokenization: $[12, 16, 43, 530, 38, 76, 15, 13, 1247, 4]$

4 Question B2: Vocabulary and Embeddings

4.1 Vocabulary & Mapping Procedure

The vocabulary was constructed by analyzing the IMDB dataset and selecting the top 10,000 most frequent tokens. Each token was mapped to a unique integer index based on frequency rank. We reserved index 0 for padding and index 1 for the start-of-sequence token.

4.2 Embedding Conversion Methods

- **One-Hot (Trainable)**: Initially randomized vectors learned during backpropagation. This serves as a dense approximation of one-hot encoding, as modern deep learning avoids sparse $10,000 \times 10,000$ matrices for computational efficiency.
- **Word2Vec**: Pre-trained vectors via the *gensim* library (Google News corpus), capturing semantic relationships via Skip-gram/CBOW.
- **GloVe**: Stanford pre-trained vectors utilizing global word-word co-occurrence statistics.

Global Parameters: Vocab: 10,000; Embedding Dim: 100; Sequence Length: 100; Hidden Units: 64.

5 Question B3: Vanilla RNN Architecture

5.1 Architecture Justification

The architecture was initially set at $L = 2$ hidden layers but later refined to $L = 1$ **with** $h = 64$.

- **Depth (L)**: Vanilla RNNs are difficult to train beyond 2 layers due to gradient instability. $L = 1$ provides stability and avoids the vanishing gradient trap where lower layer weights stop updating.
- **Dimension (h)**: 64 units provide capacity for reviews without leading to immediate overfitting.

5.2 Challenges: Vanishing Gradients

The primary challenge is the **Vanishing Gradient**. During backpropagation, gradients are multiplied by weights at every timestep. For long sequences, the gradient shrinks exponentially, causing the model to "forget" the beginning of the review. This limits the model's ability to handle **Long-Range Dependencies**.

Results (Pre-Padding):

- Epoch 1: Accuracy 0.6624, Loss 0.5929.
- Epoch 2: Accuracy 0.8381, Loss 0.3793.
- **Improved Test Accuracy: 0.8117**.

6 Question B4: Embedding Integration

6.1 Architectural Modifications

To integrate pre-trained embeddings, we modified the Embedding layer initialization:

- **Trainable (One-Hot Equivalent)**: Weights set to random; `trainable=True`.
- **Pre-trained (GloVe/Word2Vec)**: Weights set from external corpora; `trainable=False` (frozen). This significantly reduces complexity by not updating the 1M parameters in the embedding layer.

6.2 Performance Metrics and Differences

Table 1: B4 Performance Comparison

Method	Acc	Prec	F1
Trainable	0.8171	0.8163	0.8173
Static_GloVe	0.7220	0.8138	0.6744
Static_W2V	0.7214	0.6724	0.7560

6.3 Observed Differences

- **Cold Start**: Trainable models start with random noise, causing lower Epoch 1 accuracy compared to GloVe's "warm start".
- **Generalization**: GloVe/W2V yield higher recall as they understand rare words from external massive corpora.
- **Overfitting**: Trainable models are more prone to overfitting as they tune 1M additional parameters.

7 Question B5: RNN vs. LSTM Comparison

We directly compared the Vanilla RNN with LSTM using $h = 64$ and 5 epochs.

- **Training Stability**: LSTMs utilize a "Constant Error Carousel" to stabilize gradients, leading to a much lower standard deviation in validation accuracy (0.0055 vs 0.0388).
- **Gating Mechanism**: The LSTM's Forget Gate allows the network to "reset" memory, while the Cell State allows gradients to flow linearly, preventing vanishing.

Table 2: B5 Comparison Report

Metric	Vanilla RNN	LSTM
Acc	0.8002	0.8420
F1	0.8078	0.8423
Conv. Speed	23.10s	16.02s
Stability	0.0388	0.0055

8 Question B6: Attention Mechanism

8.1 Feasibility and Justification

An attention mechanism is highly feasible. It transforms the bottleneck of standard recurrent layers into a dynamic architecture that weights words by importance.

8.2 Implementation Challenges

- **Alignment**: Requires `return_sequences=True`, increasing data volume.
- **Query Selection**: Using the whole sequence as Query and Value adds complexity.
- **Vanishing Weights**: Attention weights can become sparse in very long sequences.

8.3 Computational Overhead

- **Memory**: High (linear with sequence length).
- **Time**: High ($O(N^2)$ attention matrix).

8.4 Rigorous Justification

Attention bypasses the bottleneck where semantic content is compressed into one final hidden state. It computes $C = \sum \alpha_i h_i$, allowing the model to attend to sentiment-heavy tokens regardless of their position.

9 Inference and Verification

Final experiments with Attention-LSTM (Trainable) and LSTM-GloVe showed consistent polar sentiment detection.

Table 3: Sample Verification

Snippet	Prob	Pred	Correct
"...powerful..."	0.9432	Pos	Yes
"...terrible..."	0.3870	Neg	Yes
"...girl..."	0.7991	Pos	No

10 Conclusion

This study confirms that LSTM architectures significantly outperform Vanilla RNNs in stability and long-range dependency handling. While pre-trained embeddings provide

semantic richness, trainable layers offer better adaptability to review-specific slang. Attention mechanisms provide the most robust path to accuracy by removing the recurrent bottleneck.